

ventana, dos “plot chart” que muestren la relación entre "minPorKM" y "peso", y minPorKM" y "edad".

Para comenzar, guardémonos la configuración de todos los parámetros de *par()* antes de hacer algún cambio:

```
par.original <- par(no.readonly = TRUE)
```

Hasta ahora veníamos guardando cada atributo de *par()* en forma individual, pero en realidad es más práctico guardar todos los parámetros, y restaurar toda la configuración original. Con el código anterior, nos aseguramos que guardamos toda la configuración gráfica actual.

Usemos “mfrow” para separar nuestro gráfico en un layout de 2 filas y 1 columna:

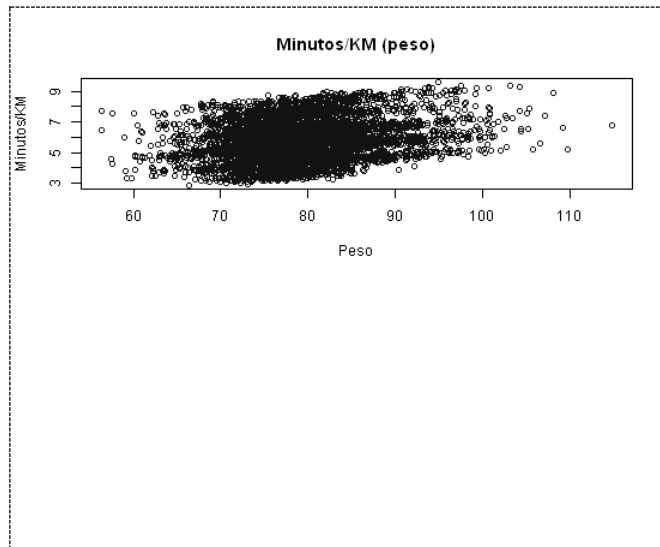
```
par(mfrow=c(2,1))
```

Al parámetro “mfrow” se le debe pasar un vector de dos componentes, donde la primera es la cantidad de filas y la segunda la cantidad de columnas. Ajustemos también el tipo de letra:

```
par(cex=0.75)
```

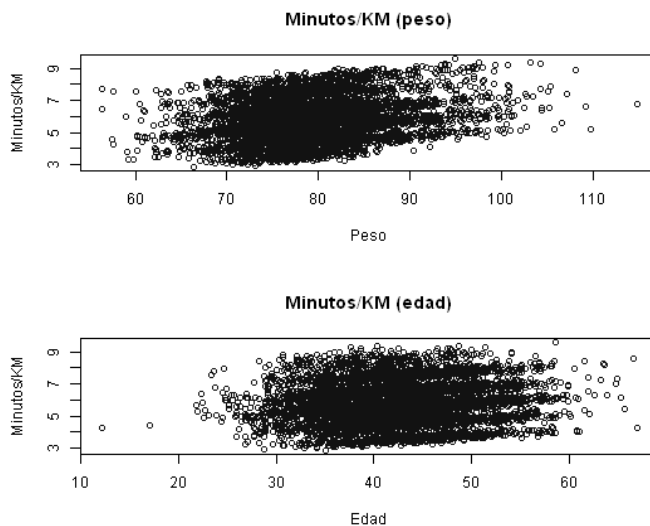
Dibujemos ahora el primer plot:

```
plot_minVsPeso <- plot(dPM$peso, dPM$minPorKM, main =  
"Minutos/KM (peso)", xlab="Peso", ylab="Minutos/KM",  
col="blue")
```



Observen como la mitad del gráfico es espacio vacío. Dibujemos ahora el segundo:

```
plot_minVsEdad <- plot(dPM$edad, dPM$minPorKM, main =
"Minutos/KM (edad)", xlab="Edad", ylab="Minutos/KM",
col="blue")
```



Restauramos ahora la configuración de `par()`:

```
par(par.original)
```

Podemos hacer diseños un poco más complejos:

```
par.original <- par(no.readonly = TRUE)

par(mfrow=c(2,2))

par(cex=0.75)

par(bg = "lightyellow")

par(bty = "n")


plot_minVsPeso <- plot(dPM$peso, dPM$minPorKM, main =
"Minutos/KM (peso)", xlab="Peso", ylab="Minutos/KM",
col="blue")

plot_minVsEdad <- plot(dPM$edad, dPM$minPorKM, main =
"Minutos/KM (edad)", xlab="Edad", ylab="Minutos/KM",
col="blue")

box_minVsSexo <- boxplot(minPorKM ~ sexo, data = dPM, col =
"gray90", border = "grey", notch=TRUE, boxwex=0.25 )

title("Vel por sexo")

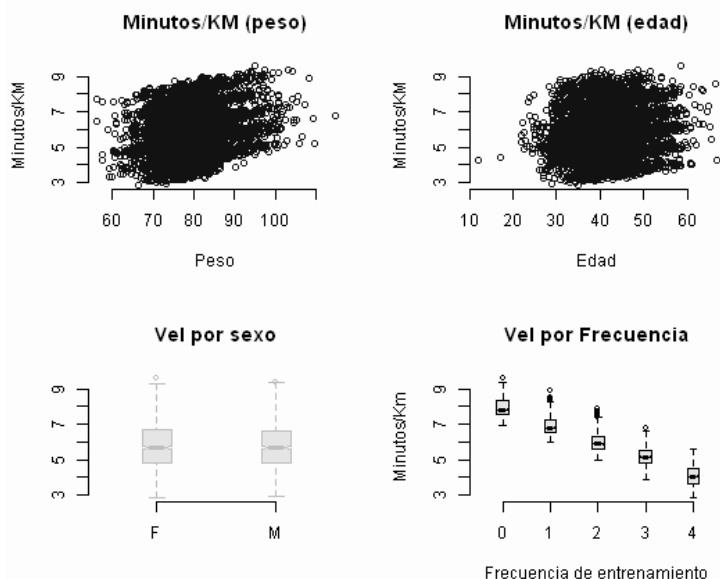

box_minVsFrecuencia <- boxplot(minPorKM ~
frecuenciaSemanalEntrenamiento, data = dPM, col = "gray90",
notch=TRUE, boxwex=0.25 )

title(main="Vel por Frecuencia")

title(xlab="Frecuencia de entrenamiento")

title(ylab="Minutos/Km")


par(par.original)
```



Un poco más a alto nivel, puedo utilizar la función *layout()*.³⁰ Primero, armo un histograma (sin graficarlo) de los campos edad y peso del data.frame dPM:

```
hist.edad <- hist(dPM$edad, plot=FALSE)
```

```
hist.peso <- hist(dPM$peso, plot=FALSE)
```

Calculo los rangos de visualización

```
top <- max(c(hist.peso$counts, hist.edad$counts))
```

```
edad.range <- c(0,max(dPM$edad))
```

```
peso.range <- c(0,max(dPM$peso))
```

Calculo los parámetros gráficos

```
grafico.ancho <- c(3,1) # Ancho relativos de las dos
columnas
```

```
grafico.alto <- c(1,3) # Alto relativo de las dos
filas
```

³⁰ El siguiente ejemplo está adaptado de <http://gallery.r-enthusiasts.com/RGraphGallery.php?graph=78>


```
grafico.nroFilas <- 2
grafico.nroColumnas <- 2
```

Armo la matriz de graficación. La misma es una matriz en la cual cada componente está asociada a la componente correspondiente del layout del gráfico y su valor indica cuál de los gráficos se graficaran en esa coordenada. Para ello, primero defino la secuencia de graficación:

```
grafico.secuenciaGraficacion <- c(2,0,1,3)
```

Estamos diciendo que, dada una matriz de 2 x 2, en (1,1) cae el gráfico número 2, en (1,2) no se grafica nada, en (2,1) va el primer grafico que hagamos y en (2,2) se dibuja el tercero. Ahora si armo la matriz:

```
matrizGraficacion <-
matrix(grafico.secuenciaGraficacion,grafico.nroFilas,
grafico.nroColumnas,byrow=TRUE)
```

Genero el layout:

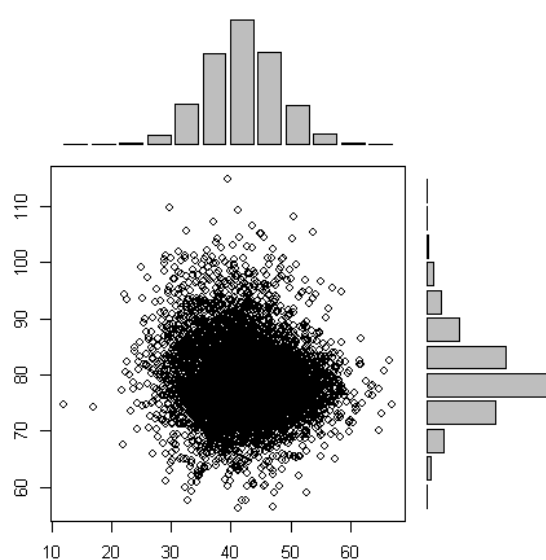
```
layout(matrizGraficacion, grafico.ancho, grafico.alto,
respect = TRUE)
```

Creo los gráficos, con "mar" defino los márgenes

```
par(mar=c(3,3,1,1))
plot(dPM$edad, dPM$peso, xlab = "edad", ylab="peso")
par(mar=c(0,3,1,1))
barplot(hist.edad$counts, axes=FALSE,)
par(mar=c(3,0,1,1))
barplot(hist.peso$counts, axes=FALSE,horiz=TRUE)
```

Restauró *par()*:

```
par(par.original)
```



Implementando nuestras primeras búsquedas

5.1. Acerca de las búsquedas por fuerza bruta

Vamos a comenzar nuestro enfoque en la optimización utilizando R para ejecutar el que quizás es el algoritmo más intuitivo de todos, la fuerza bruta. Es decir, vamos a recorrer una por una todas las alternativas posibles y evaluar cuál es la mejor solución. Supongamos que deseamos encontrar el mínimo de la función $Y=(X+10)^2 - 20$ cuando X solo puede tomar valores enteros, acotados entre -100 y 100. Definimos primero la función de evaluación:

```
funcionCuad <- function(x){  
  (-x+10)^2-20  
}
```

Y a continuación el algoritmo de fuerza bruta:

```
fuerzaBruta <- function(fn, cotaMin, cotaMax){  
  menorValor<- fn(cotaMin)  
  solucion <- cotaMin  
  for (i in (cotaMin+1):cotaMax){  
    valorActual <- fn(i)  
    if(valorActual < menorValor){  
      menorValor <- valorActual  
      solucion <- i  
    }  
  }  
  resultado <- data.frame(solucion, menorValor)  
  names(resultado) <- c("Solución", "Valor Objetivo")  
  resultado
```

```
}

fuerzaBruta(funcionCuad, -100, 100)
```

```
Solución valor objetivo
1      10      -20
```

Bueno, funciona bien, pero el código no aprovecha la expresividad de R³¹. Sabiendo que podemos usar vectorización y reciclado, que implementa los bucles “for” mucho más rápido, podemos hacer algo como esto:

```
fuerzaBruta <- function(fn, cotaMin, cotaMax, maximizar =
FALSE) {

  dominio <- seq(cotaMin, cotaMax)

  evaluacion <- fn(dominio)

  indice <- order(evaluacion, decreasing = maximizar)[1]

  resultado <- data.frame(dominio[indice],
evaluacion[indice])

  names(resultado) <- c("Solución", "Valor Objetivo")

  resultado

}

fuerzaBruta(funcionCuad, -100, 100, FALSE)
```

```
Solución valor objetivo
1      10      -20
```

Mucho más eficiente y, además, puede maximizar si así lo deseamos³². El problema anterior es muy sencillo, podemos darnos el lujo de aplicar fuerza bruta. Obviamente, este enfoque está preparado para problemas de optimización combinatoria con un número finito de soluciones. Por definición, si existen infinitas soluciones posibles, el tiempo de cómputo necesario para evaluarlas todas también es infinito. En caso de

³¹ En Python hubiéramos usado la frase “el código no es muy pythonico”. Intenté armar una frase equivalente para R, pero “eRrico” queda muy feo.

³² Y todavía se podría simplificar mucho más el código.

optimización continua, tenemos que fijar un nivel de granularidad máximo (o sea, cual es la magnitud a partir de la cual las variaciones nos parecen indiferentes). Adaptemos nuestro ejemplo para el caso continuo:

```
fuerzaBruta <- function(fn, cotaMin, cotaMax,
maximizar=FALSE, granularidad = 1){

  dominio <- seq(from= cotaMin, to= cotaMax, by=
granularidad)

  evaluacion <- fn(dominio)

  indice <- order(evaluacion, decreasing = maximizar)[1]

  resultado <- data.frame(dominio[indice],
evaluacion[indice])

  names(resultado) <- c("Solución", "Valor Objetivo")

  resultado

}

fuerzaBruta(funcionCuad, -100, 100, FALSE, 0.01)
```

	Solución	Valor Objetivo
1	10	-20

Si quisiéramos armar una versión que trabaje con múltiples variables de decisión, podríamos armar algo como esto:

```
fuerzaBruta <- function(fn, cotaMin, cotaMax,
maximizar=FALSE, granularidad = 1){

  for (i in 1:length(cotaMin)){

    if (i==1){

      auxDominio <- list(seq(from= cotaMin[i], to=
cotaMax[i], by= granularidad))

    }

    else{
```

```

        auxList <- list(seq(from= cotaMin[i], to= cotaMax[i],
by= granularidad))

        auxDominio <- c(auxDominio, auxList)

    }

}

dominio <- expand.grid(auxDominio)

evaluacion <- fn(dominio)


indice <- order(evaluacion, decreasing = maximizar)[1]

resultado <- data.frame(dominio[indice,],
evaluacion[indice,])

names(resultado) <- c(paste(rep("Variable", times =
length(cotaMin)), seq(1:length(cotaMin))), "Valor Objetivo")

resultado

}

```

Esta función requiere algo más de explicación. La estructura es muy similar a la versión de *fuerzaBruta()* que veníamos utilizando, pero con algunas diferencias sutiles. Veamos primero como trabaja. La función “fn” debe ser una función de una variable, pudiendo ser esta variable vectorial. O sea, si quiero optimizar un problema de dos variables, debería modelarlas mediante un vector de dos componentes. Las variables *cotaMin* y *cotaMax*, entonces, deben ser vectores de la misma dimensión que el vector que acepta la función *fn*. Por esta naturaliza vectorial, ya no podemos construir el dominio mediante un simple cálculo de secuencia. En vez de eso, utilizamos una variable denominada “auxDominio” la cual es una lista de tantos elementos como variables de decisión haya. La misma se rellena en el bucle “for” inicial, en la primer iteración declarándola como lista y en el resto ampliándola mediante *c()*. Elegimos usar una lista y no un data frame ya que el data frame tiene la restricciones que todos los campos deban tener la misma cantidad de registros, algo que no necesariamente se cumple. Por último, una vez generada la lista completa, en la cual cada elemento es una sublista con el dominio de cada variable independiente, usamos *expand.grid()* para

generar un data frame con el producto cartesiano de las mismas. Veamos en detalle ejecutando el siguiente código:

```
# Prueba de funcionamiento del bucle for. Se inyectan los
valores de los parámetros.
```

```
cotaMin <- c(-10,-5,-1)

cotaMax <- c(5,5,3)

granularidad <- 1

for (i in 1:length(cotaMin)){

  if (i==1){

    auxDominio <- list(seq(from= cotaMin[i], to=
cotaMax[i], by= granularidad))

  }

  else{

    auxList <- list(seq(from= cotaMin[i], to= cotaMax[i],
by= granularidad))

    auxDominio <- c(auxDominio, auxList)

  }

}

dominio <- expand.grid(auxDominio)

auxDominio

[[1]]
[1] -10 -9 -8 -7 -6 -5 -4 -3 -2 -1 0 1 2 3 4 5

[[2]]
[1] -5 -4 -3 -2 -1 0 1 2 3 4 5

[[3]]
[1] -1 0 1 2 3
```

```
dominio
```

	Var1	Var2	Var3
1	-10	-5	-1
2	-9	-5	-1
3	-8	-5	-1
4	-7	-5	-1
5	-6	-5	-1
6	-5	-5	-1

7	-4	-5	-1
8	-3	-5	-1
9	-2	-5	-1
10	-1	-5	-1
11	0	-5	-1
12	1	-5	-1
13	2	-5	-1
14	3	-5	-1
15	4	-5	-1
16	5	-5	-1
17	-10	-4	-1
18	-9	-4	-1
19	-8	-4	-1
20	-7	-4	-1
...			

Como vemos, el bucle for inicial genera todo el conjunto de soluciones a evaluar³³. El resto de la nueva función de *fuerzaBruta()* es muy similar a la versión anterior, salvo que al momento de devolver los resultados hay que poner varios nombres de variables independientes, que se hacen concatenando con *paste()* una repetición del término “Var” construida con *rep()* y el número de la variable generado con *seq()*. Probemos de ejecutarlo con nuestro problema de optimizar la función *funcionCuad()*:

```
fuerzaBruta(funcionCuad, -100, 100)
```

	variable 1	valor	Objetivo
var1	10		-20

Y probemos ahora con una función de dos variables:

```
funcion2Var <- function(x){
```

```
  x[1]*x[1] + 2*x[2]*x[2]
```

```
}
```

```
fuerzaBruta(funcion2Var, c(-100,-100), c(100,100))
```

	variable 1	variable 2	valor	Objetivo
20201	0	0		0

Grabando la función *fuerzaBruta()* a un script, podemos resolver por este método cualquier problema con variables acotadas en ambos sentidos.

³³ Por cuestiones de espacio se omitieron la mayoría de los resultados.

El gran problema con esta función es que, potencialmente, no termina nunca. Así que deberíamos definirle un parámetro con el tiempo máximo de cómputo, para que alcanzado este, termine de evaluar soluciones. Pero con esto anulamos la principal característica de la fuerza bruta, la de revisar todos los casos. Así que vamos a solucionar esto con el segundo algoritmo más intuitivo de todos, la búsqueda aleatoria.³⁴

5.2. Búsqueda aleatoria

Los algoritmos de búsqueda aleatorias se basan en el mismo principio que la fuerza bruta, pero en vez de evaluar todas las soluciones, evalúan un subconjunto seleccionado al azar. Mientras más grande sea este subconjunto, más cercano a la fuerza bruta estará este algoritmo (y más certera será la solución óptima encontrada).

Implementar un algoritmo de este tipo no requiere mucho más trabajo que el algoritmo de fuerza bruta:

```
busquedaAleatoria <- function(fn, cotaMin, cotaMax,
maximizar=FALSE, cantMuestras = 500){

  for (i in 1:length(cotaMin)){

    if (i==1){

      dominio <- data.frame(runif(cantMuestras, cotaMin[i],
cotaMax[i]))

    }

    else{

      dominio <- cbind(dominio, runif(cantMuestras,
cotaMin[i], cotaMax[i]))

    }

  }

  evaluacion <- fn(dominio)
```

³⁴ Obviamente, estas afirmaciones son muy subjetivas.

```

indice <- order(evaluacion, decreasing = maximizar)[1]

resultado <- data.frame(dominio[indice,],
evaluacion[indice,])

names(resultado) <- c(paste(rep("Variable", times =
length(cotaMin)), seq(1:length(cotaMin))), "Valor Objetivo")

resultado

}

```

Como vemos, en vez de evaluar las soluciones siguiendo una secuencia, generamos aquí valores aleatorios dentro del rango permitido para cada variable independiente. Los únicos cambios son la eliminación del parámetro granularidad, el cual es reemplazado por la cantidad de muestras, y la modificación del bucle “for” para generar valores aleatorios mediante *runif()*³⁵ en vez de todas las combinaciones posibles. Como a priori sabemos la cantidad de valores de cada variable a evaluar, y este es el mismo para todas, no necesitamos usar una lista, sino que alcanza con un data frame. A simple vista, el método es bastante ineficaz, ya que el resultado obtenido en cada evaluación no depende de evaluaciones anteriores (es decir, es un algoritmo que no mejora la solución a medida que avanza en su ejecución), además que la solución devuelta depende de los valores que haya tomado el generado de números aleatorios en esa llamada:

```
busquedaAleatoria(funcion2Var, c(-100, -10), c(100,10))
```

	Variable 1	Variable 2	Valor Objetivo
439	0.8753577	1.173358	3.519791

```
busquedaAleatoria(funcion2Var, c(-100, -10), c(100,10))
```

	Variable 1	Variable 2	Valor Objetivo
220	-2.339311	-1.06756	7.751744

```
busquedaAleatoria(funcion2Var, c(-100, -10), c(100,10))
```

	Variable 1	Variable 2	Valor Objetivo
114	-1.049637	-1.634462	6.444671

³⁵ La función *runif()* nos permite generar muestras de una variable aleatoria con distribución uniforme. Requiere de tres parámetros, que en orden son: el número de muestras a generar, la cota inferior de la distribución y la cota superior.

5.3. Búsqueda dirigida por gradientes

El caso opuesto a la búsqueda aleatoria y a la fuerza bruta es la búsqueda dirigida por “gradientes”. Aquí, a partir de una solución inicial (que puede ser generada al azar o basarse en alguna heurística particular), avanzamos de iteración en iteración “moviéndonos” en la dirección que mejor mejora nuestra función objetivo.

Si nuestra función a optimizar es diferenciable, esta dirección se puede obtener mediante el gradiente de la función. Entonces, nuestro algoritmo de búsqueda se podría implementar así:

```
busquedaPorGradiente <- function(fn, gradiente,
solucionInicial, precision, paso=1, iteraciones=100,
minimizar=TRUE){

  solActual <- solucionInicial

  for (i in 1:iteraciones){

    # Me muevo en el sentido de la variación que deseo

    if(minimizar==TRUE){

      solNueva <- solActual - paso * gradiente(solActual)

    }

    else{

      solNueva <- solActual + paso * gradiente(solActual)

    }

    #Chequeo si la variación es significativa

    if (sum((abs(solNueva - solActual))<precision)){

      break

    }

    else{

      solActual <- solNueva

    }

  }

}
```

```

    resultado <- c(solActual, fn(solActual))

    names(resultado) <- c(paste(rep("Variable", times =
length(solActual)), seq(1:length(solActual))), "Valor
Objetivo")

    resultado
}

```

Esta función requiere como parámetros obligatorios la función a optimizar, la función que genera el gradiente de la función a optimizar, la solución inicial a partir de la cual comenzar la búsqueda, y la precisión deseada.

Ya teniendo definida nuestra función de dos variables:

```

funcion2Var <- function(x){
  x[1]*x[1] + 2*x[2]*x[2]
}

```

Podemos armar una función que devuelva el gradiente de dicha función:

```

gradFuncion2Var <-function(x){
  c(2*x[1], 4*x[2])
}

```

Y la podemos probar:

```

busquedaPorGradiente(funcion2Var, gradFuncion2Var, c(-100, -
10), precision = 0.01)

```

Variable 1	Variable 2	Valor Objetivo
-1.000000e+02	-5.153775e+48	5.312280e+97

```

busquedaPorGradiente(funcion2Var, gradFuncion2Var, c(-100, -
10), precision = 0.01, paso=0.1)

```

Variable 1	Variable 2	Valor Objetivo
-4.056482e-02	-1.719071e-07	1.645505e-03

```

busquedaPorGradiente(funcion2Var, gradFuncion2Var, c(-100, -
10), precision = 0.01, paso=0.1, iteraciones=10000)

```

Variable 1	Variable 2	Valor Objetivo
-4.056482e-02	-1.719071e-07	1.645505e-03

Como suele suceder con este algoritmo, el paso debe ser de una magnitud cercana a la precisión que deseamos para llegar a resultados cercanos a algún óptimo local en un número de iteraciones aceptable.

Algoritmos para programación lineal

6.1. Introducción a la Programación Lineal

Los problemas de Programación Lineal son problemas de optimización convexa en el cual tanto la función objetivo como las restricciones son funciones lineales. Si bien su definición parece un tanto restrictiva, en realidad muchos de los problemas estudiados en el mundo real entran dentro de esta categoría, por ejemplo:

- Asignación de rutas de transporte
- Balance de líneas de producción
- Asignación de procesos a equipos de procesamiento
- Definición de mix de productos
- Definición de mezcla de componentes para productos
- Nivelación de recursos

Un problema de programación lineal se caracteriza porque su conjunto de soluciones factibles forma un politopo (o sea, la generalización de un polígono para n dimensiones). Por su propia naturaleza, al estar los gradientes definidos en una sola dirección, los puntos extremos se encuentran en la frontera del politopo, más específicamente en algunos de sus vértices. Esto permitió el desarrollo del muy eficiente Método Simplex para su resolución, que se aprovecha de esta particularidad para buscar soluciones solo en dichos puntos.

6.2. Cómo resolver problemas de PL

Dentro de los paquetes “estándares” que se distribuyen con R, tenemos el paquete “boot”. Entre otras, este paquete tiene una función denominada “simplex” que se encarga de aplicar el algoritmo de dicho nombre. Para usarla, primero carguemos el paquete:³⁶

```
library(boot)
```

³⁶ Como vimos anteriormente, los paquetes (código “empaquetado”, valga la redundancia) se cargan con *library()*, no con *source()*. Con *source()* solo cargamos código guardado en scripts

La función “simplex” se encarga de resolver problemas de programación lineal. Permite únicamente el uso de solo variables continuas y fuerza las soluciones no negativas. Veamos un ejemplo de uso:

```
coefFuncionZ <- c(1,2,3)
matrizRestriccionesMenorOIgual <-
matrix(c(0,2,3,5,0,3),2,3,byrow=TRUE)
vectorTermIndep <- c(100,100)

mySimplex <- simplex(coefFuncionZ, A1 =
matrizRestriccionesMenorOIgual, b1= vectorTermIndep,
maxi=TRUE )

mySimplex
```

Linear Programming Results

```
Call : simplex(a = coefFuncionZ, A1 =
matrizRestriccionesMenorOIgual,
b1 = vectorTermIndep, maxi = TRUE)
```

```
Maximization Problem with Objective Function Coefficients
x1 x2 x3
1 2 3
```

```
Optimal solution has the following values
```

```
x1 x2 x3
20 50 0
```

```
The optimal value of the objective function is 120.
```

Con el código anterior, resolvimos el siguiente problema:

$$\max \quad z = x_1 + 2x_2 + 3x_3$$

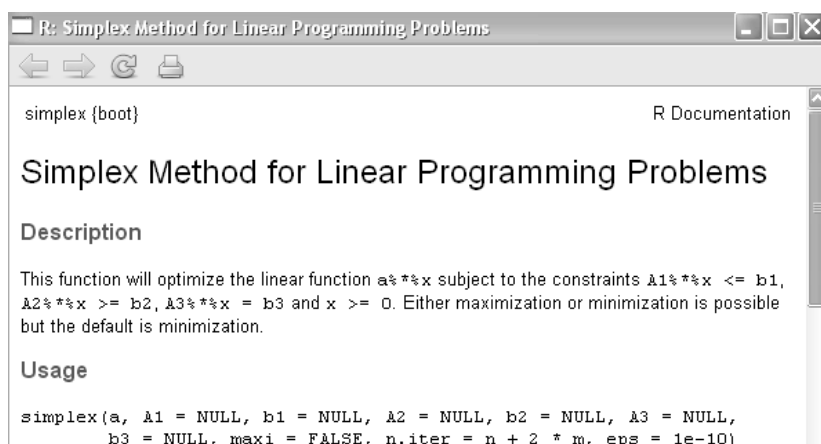
$$s.a \quad 2x_2 + 3x_3 \leq 100$$

$$5x_1 + \quad + 3x_3 \leq 100$$

$$x_1, x_2, x_3 \geq 0$$

simplex() es una función compleja para utilizar debido al gran número de parámetros que requiere. Veamos su ayuda:

```
?simplex
```



Como vemos, amén del vector "a" de los coeficientes de la función objetivo, tenemos parámetros del tipo matrix para los coeficientes de las restricciones (A1, A2 y A3, dependiendo si son del tipo $\leq$, $\geq$ o $=$, respectivamente), coeficientes para los términos independientes de las restricciones (b1, b2 y b3, con la misma lógica que los An), un parámetro maxi que si vale TRUE el solver se ejecuta maximizando (si se omite o vale FALSE se ejecuta minimizando), y parámetros de cálculo algorítmicos como "n.iter" para el número de iteraciones y "eps" para la precisión decimal al momento de comparar valores.

Entonces, como mínimo, para utilizar esta función, debemos definir un vector de coeficientes para la función objetivo, una o más matrices de coeficientes de restricciones (A1 para las restricciones $\leq$, A2 para las $\geq$, y A3 para las $=$) y el término independiente respectivo para cada tipo de restricción, y opcionalmente una indicación acerca de si queremos minimizar.

Como objeto hecho y derecho, podemos acceder a los atributos del objeto generado por simplex:

```
Class(mySimplex)
mySimplex$soln      # Soluciones
mySimplex$soln[2]   # Solución de la segunda variable
mySimplex$value     # Valor de la función objetivo
mySimplex$slack     # Holgura de cada restricción <=
mySimplex$surplus   # Excedente de cada restricción >
```


Este objeto tiene, además, otros atributos relativos al cálculo:

```
mySimplex$a          # Coeficientes de Z con variables básicas  
                      con coeficientes igual a cero  
  
mySimplex$basic      # Los índices de las variables básicas en  
                      la solución encontrada  
  
mySimplex$A          # Matriz de restricciones expresada en  
                      términos de variables no básicas
```

Esta función es de las más básicas para trabajar con problemas de optimización. No trae incorporado análisis de sensibilidad (aunque pueden calcularse), ni permite declarar variables enteras. Además presupone no negatividad, por lo cual si queremos que nuestras variable puedan tomar negativas, tenemos que utilizar el artificio de separar nuestra variable en dos³⁷. Sin embargo, es muy intuitiva en su operación con matrices. La contra más grande que tiene es que es lenta, tarda un tiempo promedio proporcional al cubo de la cantidad de restricciones para hallar una respuesta.

6.3. Otros paquetes de PL: lpSolve

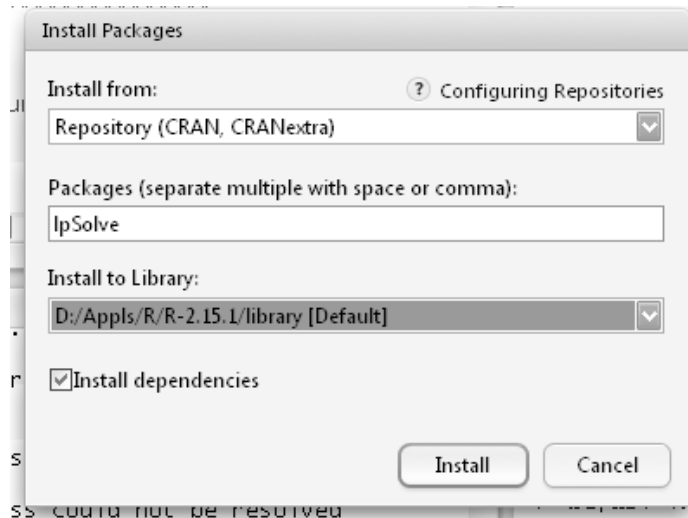
Vamos a trabajar con otro paquete para resolver problemas de optimización lineal. Como no es estándar, tenemos que descargarlo desde CRAN:

```
install.packages("lpSolve")
```

Ahora, seleccionamos la ubicación de algún servidor y listo, ya queda instalado.

En RStudio podría haber ido al menú "Tools", seleccionar el ítem "Install Packages" y escribir "lpSolve" en el cuadro de texto:

³⁷ O sea, si X_a puede tomar valores negativos, tenemos que utilizar dos variables, X_{a1} y X_{a2} , relacionándolas mediante $X_a = X_{a1} - X_{a2}$.



Una vez instalado, tenemos que cargarlo al workspace³⁸ actual con la función "library":

```
library(lpSolve)
```

lpSolve encapsula el acceso al programa también llamado lpSolve³⁹. El mismo tiene licencias LGPL y está disponible en forma de bibliotecas. El paquete lpSolve implementa algunas de las funcionalidades del programa lpSolve.

¿Se acuerdan cómo funcionaba la función "simplex" del paquete "boot"? Teníamos tres matrices de coeficientes de restricciones y tres vectores de términos independientes, uno por cada tipo de restricción ($\leq$, $=$, $\geq$). En este paquete las cosas funcionan un poco más simples. Bueno, resolvamos un problema sencillo con lpSolve.

Primero, definamos los coeficientes de la función objetivo:

```
problem.coef.obj <- c(1, 9, 3)
```

³⁸ Al espacio de trabajo actual, la sesión que tenemos abierta.

³⁹ La biblioteca se puede descargar desde: <http://lpsolve.sourceforge.net/5.5> para utilizarla en otro lenguaje de programación, pero no es necesaria si solo trabajamos con ella desde R, se instala con el comando *install.packages*

Como vemos, es una función de tres variables. Supongamos que tenemos dos restricciones, una del tipo $\geq$ y otra del tipo $\leq$:

```
problem.coef.restr <- matrix (c(1, 2, 3, 3, 2, 2), nrow=2,  
byrow=TRUE)
```

```
problem.coef.restr
```

```
      [,1] [,2] [,3]  
[1,]    1    2    3  
[2,]    3    2    2
```

En una única matriz defino mis dos restricciones. Como el parámetro “byrow” ⁴⁰vale “TRUE”, tenemos la primera restricción de coeficientes (1, 2, 3) y la segunda de coeficientes (3,-2,2).

Y ahora defino los signos de mis restricciones en un vector del tipo “character”:

```
problem.dir.restr <- c(">=", "<=")
```

```
problem.dir.restr
```

```
[1] ">=" "<="
```

Seguimos con los términos independientes:

```
problem.term.restr <- c(1, 15)
```

Maximicemos la función objetivo:

```
lp ("max", problem.coef.obj, problem.coef.restr,  
problem.dir.restr, problem.term.restr)
```

```
Success: the objective function is 67.5
```

En cuanto a sintaxis, mucho mejor que usar la función simplex. Además, el algoritmo simplex que implementa la función *lp()* es mucho más veloz.

⁴⁰ Recordemos que al crear una matriz, “byrow = TRUE” implica que pasamos los índices de la matriz por filas. En este caso, en una matriz de 2x3, los primeros 3 índices pasados corresponden a la primera fila, y los segundos 3 a la segunda fila.

Como ya sabemos, todo en R es un objeto:

```
mylp <- lp ("max", problem.coef.obj, problem.coef.restr,  
problem.dir.restr, problem.term.restr)
```

```
mylp
```

```
Success: the objective function is 67.5
```

Y como objeto, podemos acceder a sus miembros. Primero, las soluciones del problema:

```
mylp$solution
```

```
[1] 0.0 7.5 0.0
```

También el valor de Z alcanzado:

```
mylp$objval
```

```
[1] 67.5
```

¿Queremos hacer análisis de sensibilidad?, bueno, hagamos valer TRUE el parámetro "compute.sens":

```
mylp <- lp ("max", problem.coef.obj, problem.coef.restr,  
problem.dir.restr, problem.term.restr, compute.sens=TRUE)
```

Y accedamos a los atributos "sens.coef.from" y "sens.coef.to" para ver la sensibilidad de los coeficientes de la función objetivo:

```
mylp$sens.coef.from
```

```
[1] -1e+30 3e+00 -1e+30
```

```
mylp$sens.coef.to
```

```
[1] 1.35e+01 1.00e+30 9.00e+00
```

"compute.sens=TRUE" nos permite hacer análisis de dualidad también:

```
mylp$duals
```

```
[1] 0.0 4.5 -12.5 0.0 -6.0
```

Donde sí tengo "n" variables y "m" restricciones, los primeros m valores corresponden a los valores duales de las restricciones y los siguientes n valores a los duales de las actuales variables de decisión. Puedo ver la sensibilidad de los duales también:

```
mylp$duals.from
[1] -1.0e+30  1.0e+00 -1.0e+30 -1.0e+30 -1.4e+01
mylp$duals.to
[1] 1.0e+30  1.0e+30  5.0e+00  1.0e+30  7.5e+00
```

No me devuelve las holguras, pero las puedo calcular a mano. Primero devuelvo la matriz de las restricciones:

```
mylp$constraints
```

	[,1]	[,2]
	1	3
	2	2
	3	2
const.dir.num	2	1
const.rhs	1	15

La matriz se devuelve transpuesta (es la forma en que lpSolve trabaja internamente). La primera columna es la primera restricción, la segunda es la segunda restricción. La última fila representa a los términos independientes, la anteúltima son referencias al sentido de las restricciones (2 es $\geq$, 1 es $\leq$ y 3 =) y los primeros valores los coeficientes de las restricciones. Entonces, podemos hacer:

```
sum(mylp$constraints[1:3,1] * mylp$solution) -
mylp$constraints[5,1]
```

```
const.rhs
14
```

```
sum(mylp$constraints[1:3,2] * mylp$solution) -
mylp$constraints[5,2]
```

```
const.rhs
0
```

Todo muy lindo, pero con un poco de ingenio (y mucho trabajo manual) podíamos llegar a tener un análisis de sensibilidad con la función *simplex()* del paquete *boot*. ¿Qué más podemos hacer con este paquete?. Bueno, supongamos que de las tres variables queremos hacer que la segunda y la tercera sean enteras:

```
mylp <- lp ("max", problem.coef.obj, problem.coef.restr,
problem.dir.restr, problem.term.restr, compute.sens=TRUE,
int.vec=c(2:3))
```

```
mylp$solution
```

```
[1] 0.3333333 7.0000000 0.0000000
```

Con el parámetro "int.vec" puedo pasarle a lp un vector con los índices de las variables enteras. Si quiero ver cuáles eran las variables enteras:

```
mylp$int.count #Me devuelve la cantidad de variables enteras
[1] 2
mylp$int.vec # Devuelve los índices de las variables enteras
[1] 2 3
mylp$solution[mylp$int.vec] # Devuelvo los valores de las
variables enteras
[1] 7 0
```

Pero bueno, quizás quiero que la primera variable sea binaria, entonces uso el parámetro "binary.vec":

```
mylp <- lp ("max", problem.coef.obj, problem.coef.restr,
problem.dir.restr, problem.term.restr, compute.sens=TRUE,
int.vec=c(2:3), binary.vec=c(1))

mylp
mylp$solution
mylp$bin.count
mylp$binary.vec
mylp$solution[mylp$binary.vec]
```

Si tengo todas mis variables enteras, puedo reemplazar int.vec por "all.int=TRUE":

```
mylp <- lp ("max", problem.coef.obj, problem.coef.restr,
problem.dir.restr, problem.term.restr, compute.sens=TRUE,
all.int=TRUE)

mylp
mylp$solution
mylp$int.count
mylp$int.vec
mylp$solution[mylp$int.vec]
```

Y si son todas binarias, uso "all.bin=TRUE":

```
mylp <- lp ("max", problem.coef.obj, problem.coef.restr,
problem.dir.restr, problem.term.restr, compute.sens=TRUE,
all.bin=TRUE)

mylp
mylp$solution
mylp$bin.count
mylp$binary.vec
mylp$solution[mylp$binary.vec]
```

Noten que en ningún lado pusimos ninguna restricción de no negatividad. Y es que la envoltura para R de lpSolve asume que todas las variables son no negativas. Si queremos tener una variable libre, es necesario hacer la siguiente transformación: $X_n = X_{nA} - X_{nB}$, donde X_{nA} y X_{nB} son no negativas. Supongamos ahora que nuestra segunda variable puede tomar valores negativos con las variables pertenecientes al dominio real. Primero, aplico la transformación $X_n = X_{nA} - X_{nB}$ a la segunda variable en la función objetivo, duplicando el segundo coeficiente, y cambiándole el signo a la segunda duplicación:

```
problem.coef.obj <- c(1, 9, -9, 3)
```

Donde al coeficiente 9 lo separé en 9 y -9 ya que:

$$9X_2 = 9(X_{2A} - X_{2B}) = 9X_{2A} - 9X_{2B}.$$

Aplico el mismo principio en los coeficientes de las restricciones:

```
problem.coef.restr <- matrix (c(1, 2, -2, 3, 3, 2, -2, 2),
nrow=2, byrow=TRUE)

problem.coef.restr
```

```
      [,1] [,2] [,3] [,4]
[1,]    1    2   -2    3
[2,]    3    2   -2    2
```

Y a los vectores de sentidos de las restricciones y términos independientes no los modifiqué, ya que siguen igual. Ejecuto la función *lp()*:

```
mylp <- lp ("max", problem.coef.obj, problem.coef.restr,
problem.dir.restr, problem.term.restr, compute.sens=TRUE)

mylp
mylp$solution
[1] 0.0 7.5 0.0 0.0
```

El paquete lpSolve tiene algunas funcionalidades adicionales. Viene con dos funciones que implementan los algoritmos de asignación y transporte. Veamos el caso de transporte. Definamos la matriz de costo:

```
matrizCosto <- matrix (10000, 8, 5)
matrizCosto
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	10000	10000	10000	10000	10000
[2,]	10000	10000	10000	10000	10000
[3,]	10000	10000	10000	10000	10000
[4,]	10000	10000	10000	10000	10000
[5,]	10000	10000	10000	10000	10000
[6,]	10000	10000	10000	10000	10000
[7,]	10000	10000	10000	10000	10000
[8,]	10000	10000	10000	10000	10000

En esta matriz cada elemento [i,j] nos dice cual es costo unitario de transporte desde i a j. La modificaremos un poco para hacerla más interesante:⁴¹

```
matrizCosto[4,1]<- matrizCosto[-4,5]<- 0
matrizCosto
matrizCosto[1,2]<- matrizCosto[2,3]<- matrizCosto[3,4] <- 7
matrizCosto[1,3]<- matrizCosto[2,4]<- 7.7
matrizCosto
matrizCosto[5,1]<- matrizCosto[7,3]<- 8
matrizCosto
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	10000	7	7.7	10000.0	0
[2,]	10000	10000	7.0	7.7	0
[3,]	10000	10000	10000.0	7.0	0
[4,]	0	10000	10000.0	10000.0	10000
[5,]	8	10000	10000.0	10000.0	0
[6,]	10000	10000	10000.0	10000.0	0
[7,]	10000	10000	8.0	10000.0	0
[8,]	10000	10000	10000.0	10000.0	0

Sigamos modificando:

⁴¹ Observen el detalle que realizamos múltiples asignaciones en una línea. Si todas las asignaciones son del mismo valor, podemos hacer esto.


```
matrizCosto[1,4] <- 8.4
matrizCosto[6,2] <- 9
matrizCosto[8,4] <- 10
matrizCosto[4,2:4] <- c(.7, 1.4, 2.1)
matrizCosto
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,] 10000 7e+00    7.7    8.4    0
[2,] 10000 1e+04    7.0    7.7    0
[3,] 10000 1e+04 10000.0    7.0    0
[4,]      0 7e-01    1.4    2.1 10000
[5,]      8 1e+04 10000.0 10000.0    0
[6,] 10000 9e+00 10000.0 10000.0    0
[7,] 10000 1e+04    8.0 10000.0    0
[8,] 10000 1e+04 10000.0   10.0    0
```

Armemos ahora los vectores de sentido de las restricciones y el de los términos independientes, tanto para columnas como filas:

```
row.dir <- rep("<", 8)
row.term <- c(200, 300, 350, 200, 100, 50, 100, 150)
col.dir <- rep(">", 5)
col.term <- c(250, 100, 400, 500, 200)
```

Corramos ahora *lp.transport()*:

```
myTransport <- lp.transport(matrizCosto, "min", row.dir,
row.term, col.dir, col.term)
myTransport
```

Success: the objective function is 7790

```
myTransport$solution
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,]      0  100  100      0      0
[2,]      0      0  200  100      0
[3,]      0      0      0  350      0
[4,]  200      0      0      0      0
[5,]   50      0      0      0   50
[6,]      0      0      0      0   50
[7,]      0      0  100      0      0
[8,]      0      0      0   50  100
```

Muy “inteligentemente”, R nos devuelve una matriz de asignación de orígenes a destinos, en vez de una lista de valores de variables de decisión.

Definir un problema de asignación es más fácil, ya que al ser todas las restricciones " ≤ 1 " con variables binarias, lpSolve me automatiza la carga, por lo cual solo me ocupo de la matriz de costo:

```
matrizCostoAsignacion <- matrix (c(2, 7, 7, 2, 7, 7, 3, 2,
7, 2, 8, 10, 1, 9, 8, 2), 4, 4)

matrizCostoAsignacion
```

	[,1]	[,2]	[,3]	[,4]
[1,]	2	7	7	1
[2,]	7	7	2	9
[3,]	7	3	8	8
[4,]	2	2	10	2

```
myAssign <- lp.assign(matrizCostoAsignacion, "min")
myAssign
```

Success: the objective function is 8

```
myAssign$solution
```

	[,1]	[,2]	[,3]	[,4]
[1,]	0	0	0	1
[2,]	0	0	1	0
[3,]	0	1	0	0
[4,]	1	0	0	0

6.4. Otros paquetes de PL: lpSolveAPI

lpSolve no es la única implementación de "lpSolve" para R, tenemos también lpSolveAPI:

```
install.packages("lpSolveAPI")
library(lpSolveAPI)
```

Los modelos con lpSolveAPI se construyen de otra forma. Para empezar, no son objetos R hechos y derechos, así que es muy parecido su uso a como se usaría desde C.

Con lpSolveAPI comienzo así:

```
lprec <- make.lp(0, 4)
lprec
```

Model name:

	C1	C2	C3	C4
Minimize	0	0	0	0
Kind	Std	Std	Std	Std
Type	Real	Real	Real	Real
Upper	Inf	Inf	Inf	Inf
Lower	0	0	0	0

Con esto creo un objeto "modelo lineal" con 4 variables de decisión.
Definamos ahora los coeficientes:

```
set.objfn(lprec, c(1, 3, 6.24, 0.1))
lprec
```

Model name:				
	C1	C2	C3	C4
Minimize	1	3	6.24	0.1
Kind	Std	Std	Std	Std
Type	Real	Real	Real	Real
Upper	Inf	Inf	Inf	Inf
Lower	0	0	0	0

Agrego las restricciones:

```
add.constraint(lprec, c(0, 78.26, 0, 2.9), ">=", 92.3)
add.constraint(lprec, c(0.24, 0, 11.31, 0), "<=", 14.8)
add.constraint(lprec, c(12.68, 0, 0.08, 0.9), ">=", 4)
lprec
```

Model name:					
	C1	C2	C3	C4	
Minimize	1	3	6.24	0.1	
R1	0	78.26	0	2.9	>= 92.3
R2	0.24	0	11.31	0	<= 14.8
R3	12.68	0	0.08	0.9	>= 4
Kind	Std	Std	Std	Std	
Type	Real	Real	Real	Real	
Upper	Inf	Inf	Inf	Inf	
Lower	0	0	0	0	

En caso de restricciones cuya única función sea acotar una única variable, las agrego separadamente como "Bounds":

```
set.bounds(lprec, lower = c(28.6, 18), columns = c(1, 4))
set.bounds(lprec, upper = 48.98, columns = 4)
lprec
```

Model name:					
	C1	C2	C3	C4	
Minimize	1	3	6.24	0.1	
R1	0	78.26	0	2.9	>= 92.3
R2	0.24	0	11.31	0	<= 14.8
R3	12.68	0	0.08	0.9	>= 4

Kind	Std	Std	Std	Std
Type	Real	Real	Real	Real
Upper	Inf	Inf	Inf	48.98
Lower	28.6	0	0	18

Puedo ponerle nombre:

```
rowNames <- c("Rest. 01", "Rest. 02", "Rest. 03")
colNames <- c("X1", "X2", "X3", "X4")
dimnames(lprec) <- list(rowNames, colNames)
lprec
```

Model name:					
	X1	X2	X3	X4	
Minimize	1	3	6.24	0.1	
Rest. 01	0	78.26	0	2.9	>= 92.3
Rest. 02	0.24	0	11.31	0	<= 14.8
Rest. 03	12.68	0	0.08	0.9	>= 4
Kind	Std	Std	Std	Std	
Type	Real	Real	Real	Real	
Upper	Inf	Inf	Inf	48.98	
Lower	28.6	0	0	18	

Listo, ahora puedo resolverlo:

```
solve(lprec)
```

```
[1] 0
```

Si quiero ver el valor de Z:

```
get.objective(lprec)
```

```
[1] 31.78276
```

Las soluciones:

```
get.variables(lprec)
```

```
[1] 28.60000 0.00000 0.00000 31.82759
```

Los valores de las restricciones:

```
get.constraints(lprec)
```

```
[1] 92.3000 6.8640 391.2928
```

Si quiero que mi cuarta variable tome valores enteros:

```
set.type(lprec, 4, "integer")
```

```
lprec
```

Model name:

	x1	x2	x3	x4		
Minimize	1	3	6.24	0.1		
Rest. 01	0	78.26	0	2.9	>=	92.3
Rest. 02	0.24	0	11.31	0	<=	14.8
Rest. 03	12.68	0	0.08	0.9	>=	4
Kind	Std	Std	Std	Std		
Type	Real	Real	Real	Int		
Upper	Inf	Inf	Inf	48.98		
Lower	28.6	0	0	18		

```
solve(lprec)
```

```
get.objective(lprec)
```

```
[1] 31.792
```

```
get.variables(lprec)
```

```
[1] 28.60000000 0.03066701 0.00000000 31.00000000
```

```
get.constraints(lprec)
```

```
[1] 92.300 6.864 390.548
```

Si la segunda variable debiera ser binaria⁴²:

```
set.type(lprec, 2, "binary")
```

```
lprec
```

Model name:

	x1	x2	x3	x4		
Minimize	1	3	6.24	0.1		
Rest. 01	0	78.26	0	2.9	>=	92.3
Rest. 02	0.24	0	11.31	0	<=	14.8
Rest. 03	12.68	0	0.08	0.9	>=	4
Kind	Std	Std	Std	Std		
Type	Real	Int	Real	Int		
Upper	Inf	1	Inf	48.98		
Lower	28.6	0	0	18		

```
solve(lprec)
```

```
get.objective(lprec)
```

⁴² Observen como la declara entera y acotada por arriba en 1 y por abajo en 0.

```
[1] 31.8
```

```
get.variables(lpvec)
```

```
[1] 28.6 0.0 0.0 32.0
```

Lo bueno del paquete lpSolveAPI es que permite leer y grabar archivos estándares de programación lineal en formato “mps”:

```
write.lp(lpvec, "..\\data\\lpModel01.mps", type = "mps")
```

Vaciamos la memoria de trabajo:

```
rm(list=ls())
```

Y carguemos nuestro modelo:

```
lpvec <- read.lp("..\\data\\lpModel01.mps", type = "mps")  
lpvec
```

6.5. Optimizando en base a muestras

Pero pese a todo, la potencia de R no radica en poder ejecutar un simplex, sino en cómo lo puede complementar. En un problema real, nuestro trabajo no comienza con una definición del problema en forma de ecuaciones, sino con un conjunto de datos de relevamientos que hay que procesar en forma estadística. Resolvamos por ejemplo, un problema de mezcla apelando a la función *simplex()*.

Supongamos que tenemos todos nuestros relevamientos en un archivo en formato delimitado por comas denominado “problemaMezclaMuestras.csv”⁴³:

Con *read.csv()* lo leo y lo guardo en un data frame:

```
setwd("D://Proyectos//IOR//scripts")
```

⁴³ Lo puedo descargar de

<http://www.modelizandosistemas.com.ar/p/optimizacion-con-r.html>, y debo grabarlo en el directorio “data” dentro de mi directorio de proyectos.

```
misMuestras <- read.csv("../data//problemaMezcla-
Muestras.csv")
```

Con `View()` y `str()` puedo analizarlo:

```
str(misMuestras)
```

```
'data.frame':    106 obs. of  10 variables:
 $ propCompA : num  0.3112 0.1444 0.2688 0.0634 0.409 ...
 $ propCompB : num  0.0656 0.2818 0.2571 0.2501 0.2185 ...
 $ propCompC : num  0.2511 0.2873 0.2083 0.246 0.0904 ...
 $ propCompD : num  0.372 0.286 0.266 0.44 0.282 ...
 $ propResiduoA: num  0.15 0.174 0.15 0.202 0.14 ...
 $ propResiduoB: num  0.1621 0.1321 0.1389 0.0806 0.1977 ...
 $ propResiduoC: num  0.148 0.137 0.189 0.155 0.213 ...
 $ propResiduoD: num  0.173 0.237 0.25 0.253 0.232 ...
 $ propResiduoE: num  0.0987 0.076 0.0889 0.0659 0.0594 ...
 $ propProducto: num  0.268 0.244 0.182 0.244 0.158 ...
```

```
View(misMuestras)
```

	propCompA	propCompB	propCompC	propCompD	propResiduoA	propResiduoB	propResiduoC	propResiduoD	propResiduoE	propProducto
1	0.3112	0.0656	0.2511	0.3721	0.150365	0.162097	0.147827	0.172745	0.098729	0.268237
2	0.1444	0.2818	0.2873	0.2865	0.174249	0.132121	0.136621	0.237125	0.075951	0.243933
3	0.2688	0.2571	0.2083	0.2658	0.150362	0.138926	0.188931	0.250380	0.088921	0.182480
4	0.0634	0.2501	0.2460	0.4405	0.201559	0.080565	0.155320	0.252615	0.065948	0.243993
5	0.4090	0.2185	0.0904	0.2821	0.139863	0.197653	0.212788	0.232335	0.059408	0.157953
6	0.2959	0.3101	0.1719	0.2221	0.162242	0.150503	0.206043	0.250125	0.083711	0.147376
7	0.0968	0.3454	0.4189	0.1389	0.149345	0.096253	0.155873	0.323105	0.093571	0.181853
8	0.3910	0.2804	0.2362	0.0924	0.136822	0.200264	0.207194	0.241160	0.097498	0.117062
9	0.2666	0.2133	0.3290	0.1911	0.160849	0.149015	0.156110	0.258665	0.089166	0.186195
10	0.1558	0.3032	0.3437	0.1973	0.161627	0.117849	0.181489	0.295785	0.108815	0.134435
11	0.1800	0.0650	0.3885	0.3665	0.201445	0.168945	0.107345	0.186525	0.080995	0.254745
12	0.1926	0.4788	0.1900	0.1386	0.150684	0.107510	0.198580	0.323590	0.080296	0.139340
13	0.1482	0.0458	0.6845	0.1215	0.152457	0.172675	0.108265	0.190695	0.113119	0.262789

Y tomemos ahora el archivo “problemaMezcla-Restricciones.csv”⁴⁴ en el cual tengo tabuladas mis restricciones.

```
misRestricciones <- read.csv("../data//problemaMezcla-
Restricciones.csv")
```

```
str(misRestricciones)
```

```
'data.frame':    1 obs. of  5 variables:
 $ cantMaxResA: num 0.25
 $ cantMaxResB: num 0.3
 $ cantMaxResC: num 0.3
 $ cantMaxResD: num 0.1
 $ cantMaxResE: num 0.15
```

```
View(misRestricciones)
```

⁴⁴ Lo puedo descargar de

<http://www.modelizandosistemas.com.ar/p/optimizacion-con-r.html>, y debo grabarlo en el directorio “data” dentro de mi directorio de proyectos.

1 observations of 5 variables					
	cantMaxResA	cantMaxResB	cantMaxResC	cantMaxResD	cantMaxResE
1	0.25	0.3	0.3	0.1	0.15

Supongamos que tengo las muestras anteriores, las que indican que proporción de cada componente se agregó en la mezcla, y que proporción de productos y residuos se obtuvo:

```
componentes <- c(seq(1:4))
residuos <- c(seq(1:5) + 4)
producto <- 10
View(misMuestras[componentes])
View(misMuestras[residuos])
View(misMuestras[producto])
```

Chequeo que las proporciones estén normalizadas (o sea, sumen 1):

```
summary(misMuestras[1]+misMuestras[2]+misMuestras[3]+misMuestras[4])
```

```
propCompA
Min. :1
1st Qu.:1
Median :1
Mean :1
3rd Qu.:1
Max. :1
```

```
summary(misMuestras[5]+misMuestras[6]+misMuestras[7]+misMuestras[8]+misMuestras[9]+misMuestras[10])
```

```
propResiduoA
Min. :1
1st Qu.:1
Median :1
Mean :1
3rd Qu.:1
Max. :1
```

¿Cómo meto esto en un simplex? Bueno, en principio, no puedo. Pero lo que puedo hacer es estimar la relación entre cada componente con cada residuo y con el producto:

```
lm.propResiduoA <- lm (propResiduoA ~ 0 + propCompA +
propCompB + propCompC + propCompD, data = misMuestras)

lm.propResiduoB <- lm (propResiduoB ~ 0 + propCompA +
propCompB + propCompC + propCompD, data = misMuestras)
```



```
lm.propResiduoC <- lm (propResiduoC ~ 0 + propCompA +
propCompB + propCompC + propCompD, data = misMuestras)
lm.propResiduoD <- lm (propResiduoD ~ 0 + propCompA +
propCompB + propCompC + propCompD, data = misMuestras)
lm.propResiduoE <- lm (propResiduoE ~ 0 + propCompA +
propCompB + propCompC + propCompD, data = misMuestras)
lm.propProducto <- lm (propProducto ~ 0 + propCompA +
propCompB + propCompC + propCompD, data = misMuestras)
```

Con los ceros al inicio de la lista de variables independientes me aseguro de no incluir el término independiente. Veamos los ajustes que realizó lm:

```
summary(lm.propResiduoA)
summary(lm.propResiduoB)
summary(lm.propResiduoC)
summary(lm.propResiduoD)
summary(lm.propResiduoE)
summary(lm.propProducto)
```

Armemos nuestro vector de coeficientes de la función objetivo:

```
coefFuncionObjetivo <- lm.propProducto$coefficients[1:4]
coefFuncionObjetivo
```

propCompA	propCompB	propCompC	propCompD
0.14700694	-0.02702092	0.26196634	0.39491503

Las restricciones son todas del tipo $\leq$, así que tenemos que armar A1.

```
aux_coeficientesA1 <-
c(lm.propResiduoA$coefficients[1:4], lm.propResiduoB$coefficients[1:4] )

aux_coeficientesA1 <-
c(aux_coeficientesA1, lm.propResiduoC$coefficients[1:4], lm.propResiduoD$coefficients[1:4] )

aux_coeficientesA1 <-
c(aux_coeficientesA1, lm.propResiduoE$coefficients[1:4] )

restriccionesA1 <- matrix(aux_coeficientesA1, 5, 4,
byrow=TRUE)
restriccionesA1
```

```

      [,1]      [,2]      [,3]      [,4]
[1,] 0.06012122 0.14674360 0.15009861 0.27783015
[2,] 0.33830113 0.02659127 0.14828142 0.07405588
[3,] 0.27897496 0.28152555 0.06114977 0.07323397
[4,] 0.11694078 0.53158161 0.23098814 0.09057579
[5,] 0.05865496 0.04057890 0.14751572 0.08938917

```

Armamos los términos independientes (b1).

```

termIndepb1 <- as.numeric(c(misRestricciones[1,]))
termIndepb1

```

```
[1] 0.25 0.30 0.30 0.10 0.15
```

Ojo, tengo una restricción del tipo =, para asegurarme que las proporciones de los componentes sumen 1. Esto lo cargo en la matriz A3:

```

restriccionesA3 <- matrix(c(1,1,1,1),1,4,byrow=TRUE)
termIndepb3 <- 1

```

Y ahora ejecuto *simplex()*:

```

simplexProblemaMezcla <- simplex(coefFuncionObjetivo,
A1=restriccionesA1, b1=termIndepb1, A3=restriccionesA3,
b3=termIndepb3, maxi=TRUE )
simplexProblemaMezcla

```

Linear Programming Results

```

Call : simplex(a = coefFuncionObjetivo, A1 = restriccionesA1, b1 =
termIndepb1, A3 = restriccionesA3, b3 = termIndepb3, maxi = TRUE)

```

Maximization Problem with Objective Function Coefficients

```

      x1      x2      x3      x4
0.14700694 -0.02702092 0.26196634 0.39491503

```

Optimal solution has the following values

```

      x1      x2      x3      x4
0.09940407 0.00000000 0.04845317 0.85214276

```

The optimal value of the objective function is 0.363830172772084.

```
simplexProblemaMezcla$slack
```

```
[1] 0.00000000 0.19608060 0.20690005 0.00000000 0.06084952
```


Algoritmos para programación no lineal

7.1. La función *OPTIM*

Existe una función llamada *optim()*, dentro del paquete stats (el cual se carga por defecto y no es necesario instalarlo), que sirve para optimizar funciones lineales y no lineales, a la cual hay que pasarle como parámetro la función a optimizar de esta forma:

```
myCuadratica <- function(x){
  x^2
}
optim(par=100, fn=myCuadratica)

$par
[1] 0
$value
[1] 0
$counts
function gradient
      32      NA
$convergence
[1] 0
$message
NULL
Mensajes de aviso perdidos
In optim(par = 100, fn = myCuadratica) :
  one-diml optimization by Nelder-Mead is unreliable:
use "Brent" or optimize() directly
```

optim() es una función que permite minimizar funciones no lineales, por varios métodos. Los dos parámetros principales son "fn", el cual es la función a optimizar, y "par", que representa al valor inicial desde el cual queremos comenzar la búsqueda. Son los únicos dos parámetros obligatorios, y si los pongo en orden puedo prescindir de "nombre=":

```
optim(100, myCuadratica)

$par
[1] 0
$value
```